

Network Data format

The format in which network data is stored and retrieved depends on the packages you are using. **Igraph**, **statnet**, and the like all have their own way of storing network data and to use them, you will have to become familiar with reformatting your data for each specific package.

However, there are similarities in the concepts underlying the design of each of the format. We will examine those first before moving on to look at the specific requirements of three popular packages.

Two fundamental network data format

When it comes to data formats, in theory, there are only two types of representations that you will encounter again and again. It is important to understand the differences between them and their strenght and limitations.

Matrices

The most common network format is a matrix. Either a square matrix recording ties between the same people in column and rows; or a rectangular matrix storing affiliation of people in rows to organizations or events in column. The first type of network is usually referred to as a 1-mode network, while the second is known as a 2-mode network.

Let's have a look at some simple examples:

```
net <- matrix(c(0, 0, 1, 0, 0, 1, 1, 1, 0), nrow = 3, ncol = 3, byrow = TRUE)
net
```

```
##      [,1] [,2] [,3]
## [1,]    0    0    1
## [2,]    0    0    1
## [3,]    1    1    0
```

In such a case, you can see that the matrix is symmetric. 1 and 2 are connected to 3 and you can see evidence of this both on the third line and on the third column. Now, think about a situation in which you would ask people to nominate who are their friends. In this case, the matrix need not be symmetric. Ben can nominate Simon, even if Simon do not mention Ben.

```
net <- matrix(c(0, 1, 1, 0, 0, 0, 1, 0, 0), nrow = 3, ncol = 3, byrow = TRUE,
```

```

dimnames = list(row = c("Ben", "Simon", "William"), col = c("Ben", "Simon",
"William")))
net

##           col
## row      Ben Simon William
## Ben       0     1      1
## Simon     0     0      0
## William   1     0      0

```

The following matrix records the “affiliation” of two people to restaurant (by that I mean whether or not they have had dinner there).

```

net <- matrix(c(0, 1, 1, 0, 1, 0), nrow = 2, ncol = 3, byrow = TRUE, dimnames = list(row = c(
"George"), col = c("Racine", "The four star", "Queen's Arms")))
net

##           col
## row      Racine The four star Queen's Arms
## Andy       0           1           1
## George     0           1           0

```

Matrices are very convenient, when they are small. Good luck figuring out what is going on in a large matrix. In addition, large matrix are expensive to hold in memory: a square matrix with a thousand rows and columns has a million cells. If the matrix is sparse, meaning each individual has relations with a few other individuals where this number is much smaller than the number of individual in the network (if d is the average degree of each node and N the number of nodes, a sparse network is a network in which $d \ll N$), then a matrix representation becomes unnecessarily memory hungry.

Edgelist

Edgelist can help us overcome the limitation of the matrix representation of networks. An edgelist is a list of links between the nodes of the network. The biggest advantage of that representation is that the edgelist weight in memory increases linearly with the number of links in the network (instead of as the square of the number of nodes for a matrix representation).

For that reason, edgelist are the only alternative we have when working with very large networks (hundred of thousands to millions of links).

A simple edgelist would look like the following example.

```
edgelist <- data.frame(senders = c("Tommy", "George", "Raymond", "Thomas", "Maryam",
  "Sam", "Emily", "Jenny", "Raymond", "Maryam", "Emily", "Tommy", "Thomas",
  "Thomas"), receivers = c("Sam", "Thomas", "Emily", "Jenny", "Thomas", "Maryam",
  "Tommy", "Emily", "Thomas", "George", "Raymond", "Raymond", "Maryam", "Tommy"))
edgelist
```

```
##   senders receivers
## 1   Tommy       Sam
## 2   George    Thomas
## 3 Raymond     Emily
## 4   Thomas     Jenny
## 5   Maryam    Thomas
## 6     Sam     Maryam
## 7   Emily     Tommy
## 8   Jenny     Emily
## 9 Raymond    Thomas
## 10 Maryam    George
## 11   Emily    Raymond
## 12   Tommy    Raymond
## 13 Thomas    Maryam
## 14 Thomas     Tommy
```

As you can see, the list is only as long as the number of ties. This has one drawback, however: how does one record isolates? There is no one best way to do it. Either you can record nodes in the network through a loop (a node tied to itself), or you need a separate list of nodes for the network.

Statnet

In statnet, a network is represented as a matrix or an edgelist. But the underlying functions often use edgelists for efficiency purposes.

The main functions to built networks is `network()` which can accept data in the form of an edgelist or a matrix.

```
library(statnet)
```

```
net <- network(edgelist)
# This has transformed our edgelist in a network You can see detail of that
# network by using summary() or str()
summary(net)
```

```
## Network attributes:
##   vertices = 8
```

```

## directed = TRUE
## hyper = FALSE
## loops = FALSE
## multiple = FALSE
## bipartite = FALSE
## total edges = 14
## missing edges = 0
## non-missing edges = 14
## density = 0.25
##
## Vertex attributes:
## vertex.names:
## character valued attribute
## 8 valid vertex names
##
## No edge attributes
##
## Network adjacency matrix:
##      Emily George Jenny Maryam Raymond Sam Thomas Tommy
## Emily      0      0      0      0      1      0      0      1
## George      0      0      0      0      0      0      1      0
## Jenny       1      0      0      0      0      0      0      0
## Maryam      0      1      0      0      0      0      1      0
## Raymond     1      0      0      0      0      0      1      0
## Sam         0      0      0      1      0      0      0      0
## Thomas      0      0      1      1      0      0      0      1
## Tommy       0      0      0      0      1      1      0      0

net2 <- network(matrix(c(0, 1, 1, 0, 0, 0, 1, 0, 0), nrow = 3, ncol = 3, byrow = TRUE,
  dimnames = list(row = c("Ben", "Simon", "William"), col = c("Ben", "Simon",
    "William"))))

summary(net2)

## Network attributes:
## vertices = 3
## directed = TRUE
## hyper = FALSE
## loops = FALSE
## multiple = FALSE
## bipartite = FALSE
## total edges = 3
## missing edges = 0
## non-missing edges = 3

```

```
## density = 0.5
##
## Vertex attributes:
##   vertex.names:
##     character valued attribute
##     3 valid vertex names
##
## No edge attributes
##
## Network adjacency matrix:
##           Ben Simon William
## Ben           0      1      1
## Simon          0      0      0
## William        1      0      0
```

Igraph

Igraph is constructed for efficiency, which means that all the underlying routines are coded in C. This is great in terms of efficiency but it also makes igraph objects a bit more difficult to work with. The main function to input network data in igraph is `graph`. with the type of data you are inputting after the `.` (one of either “edgelist”, “data.frame”, “adjacency”).

```
library(igraph)
```

```
net <- graph.edgelist(as.matrix(edgelist)) #edgelist is in fact a data frame.
summary(net)
```

```
## IGRAPH DN-- 8 14 --
## attr: name (v/c)
```

```
str(net)
```

```
## IGRAPH DN-- 8 14 --
## + attr: name (v/c)
## + edges (vertex names):
## [1] Tommy ->Sam      George ->Thomas  Raymond->Emily   Thomas ->Jenny
## [5] Maryam ->Thomas  Sam      ->Maryam  Emily  ->Tommy   Jenny  ->Emily
## [9] Raymond->Thomas  Maryam ->George  Emily  ->Raymond Tommy  ->Raymond
## [13] Thomas ->Maryam  Thomas ->Tommy
```

```
net2 <- graph.adjacency(matrix(c(0, 1, 1, 0, 0, 0, 1, 0, 0), nrow = 3, ncol = 3,
```

```

      byrow = TRUE, dimnames = list(row = c("Ben", "Simon", "William"), col = c("Ben",
      "Simon", "William"))))
summary(net2)

## IGRAPH DN-- 3 3 --
## attr: name (v/c)

str(net2)

## IGRAPH DN-- 3 3 --
## + attr: name (v/c)
## + edges (vertex names):
## [1] Ben      ->Simon  Ben      ->William William->Ben

net3 <- graph.data.frame(edgelist)
summary(net3)

## IGRAPH DN-- 8 14 --
## attr: name (v/c)

str(net3)

## IGRAPH DN-- 8 14 --
## + attr: name (v/c)
## + edges (vertex names):
## [1] Tommy ->Sam      George ->Thomas Raymond->Emily Thomas ->Jenny
## [5] Maryam ->Thomas Sam      ->Maryam Emily ->Tommy Jenny ->Emily
## [9] Raymond->Thomas Maryam ->George Emily ->Raymond Tommy ->Raymond
## [13] Thomas ->Maryam Thomas ->Tommy

```